AI ENGINEERING / 01 | FOUNDATIONS

Models, tokens, prompting, and tool-driven applications

AI engineering — Builds reliable software around machine-learning models. It covers requirements, model selection, data, prompts, application logic, evaluation, deployment, and monitoring. The goal is a useful, measurable product, not simply a sophisticated demonstration or a large model.

Foundation models and LLMs — A foundation model is pretrained on broad data and reused for many tasks. A large language model predicts token sequences to summarize, extract, classify, explain, and generate text. Fluency is not proof of correctness; factual answers still need validation.

Tokens and tokenization — Tokenization splits input into model-specific units. A word can use several tokens, and whitespace or punctuation affects the count. Context limits, input and output charges, and maximum generation settings are generally expressed in tokens, not characters.

Context window — The finite token budget containing instructions, user messages, retrieved passages, tool results, and output. Long histories and excessive documents raise costs and can bury important evidence. Good context management keeps relevant information while removing duplication and stale details.

Training and inference — Training adjusts model weights by optimizing on data; inference produces outputs using the existing weights. Most AI products call pretrained models through APIs or local servers. Adding text to a prompt changes the current context, not the model's learned parameters.

Parameters and decoding — Weights are learned values; inference settings control response behavior. Decoding selects successive tokens, while temperature and top-p influence sampling diversity when supported. Maximum output tokens limits response length. Settings differ by provider and must be tested rather than copied blindly.

Prompt engineering — Crafts task instructions, context, constraints, examples, and expected output. Strong prompts say what the model should do, which evidence to use, how to handle uncertainty, and what format to return. Evaluate prompts on diverse inputs and difficult edge cases.

Instruction hierarchy — System and developer instructions define application behavior; users provide task requests; retrieved documents and webpages supply data. Treat external text as untrusted content, even when it contains instructions. This separation helps prevent accidental overrides and prompt injection.

Zero-shot and few-shot prompting — Zero-shot prompting supplies instructions without demonstrations; few-shot prompting adds selected input-output examples. Examples can establish a tone, label format, or extraction pattern. Include tricky and representative cases, not just easy examples that hide failure modes.

Structured outputs — Applications often need validated JSON or another defined schema rather than prose. Specify field names, types, required values, and permitted categories; then check the returned data in code. A well-formed response can still be semantically wrong.

Tool and function calling — A model may propose a function name and arguments for searching, calculating, or reading a database. The backend decides which tools are available, validates arguments, checks permissions, executes operations, and returns results. Generated tool requests are never authorization by themselves.

Agents and multimodal workflows — An agent can choose steps, call tools, inspect results, and iterate toward a goal. Bound its time, budget, and actions; simple workflows often work better. Multimodal systems extend inputs to images, sound, or video but still require preprocessing and task-specific testing.

AI ENGINEERING / 02 | RAG & MODEL ADAPTATION

Document retrieval, vector search, fine-tuning, and grounding

Retrieval-augmented generation (RAG) — Retrieves relevant external passages before asking a model to answer. It can incorporate changing documents and support source-based responses without retraining weights. Accuracy still depends on ingestion, retrieval, prompt assembly, and whether the model faithfully uses retrieved material.

Document ingestion — Extract readable text from PDFs, websites, databases, and uploaded files. Preserve document IDs, headings, dates, permissions, and source locations as metadata. Repair broken text extraction and duplicate material early: incorrectly parsed content creates retrieval and answer errors downstream.

Chunking and overlap — Split documents into retrievable passages. A chunk size such as 500 may represent tokens, words, or characters depending on the implementation. Small chunks improve precision; large chunks preserve context. Overlap retains information across boundaries but creates storage and retrieval redundancy.

Embeddings — An embedding model transforms text into a vector of numbers useful for semantic comparison. Vector dimension is set by the embedding model or its supported dimensionality option, not by sentence length. Query and document vectors must use compatible models and preprocessing.

Vector stores — Persist embeddings together with the source passage, document identifier, and metadata. FAISS, Chroma, and hosted vector databases support different storage and indexing patterns. A real system also needs access control, updates, deletion, and backups, not merely similarity search.

Similarity and ANN search — Cosine similarity measures alignment between vectors; dot product and Euclidean distance are alternatives. Approximate nearest neighbor indexes, such as HNSW, speed search with tradeoffs in memory and recall. A high vector score does not guarantee the passage answers a question.

Top-k retrieval — Top-k selects the number of passages to return, for example the five most similar chunks. More candidates may help recall but can waste context on irrelevant evidence. Choose k using test questions, retrieval recall, downstream answer accuracy, and latency.

Metadata filtering — Restrict candidates using hard fields such as language, document type, publication date, product stock status, or tenant. Semantic similarity is not reliable for enforcing these constraints. Apply permission filters before results reach the model rather than relying on natural-language instructions.

Hybrid retrieval — Combines keyword matching with semantic vector search. Keywords help with exact identifiers, rare names, and codes; embeddings help with paraphrases. Merging both rankings can improve recall. Good hybrid search is judged on the actual question distribution, not assumed superior everywhere.

Reranking — A second-stage model scores a short list of retrieved passages against the whole query and reorders the list. It may improve precision after fast initial retrieval, but adds computation, cost, and latency. Compare improvements using a held-out evaluation set.

Grounding, context, and hallucination — Selected passages enter an evidence-aware prompt with clear source references and token limits. Grounding asks the model to answer from verifiable evidence; hallucinations are unsupported claims. RAG reduces some hallucinations but cannot guarantee truth, accurate citations, or correct source interpretation.

Fine-tuning, LoRA, and QLoRA — Fine-tuning changes behavior through training examples; LoRA trains lightweight adapters, and QLoRA combines adapters with quantized base weights to reduce memory use. Use RAG for changing knowledge and citations, fine-tuning for consistent task behavior, or both when evidence and specialized output matter.

AI ENGINEERING / 03 | PRODUCTION & TRUST

Performance, evaluation, security, reliability, and operations

System architecture and APIs — A deployed product connects a UI, backend, model service, database, retrieval components, and monitoring. Model APIs differ in supported parameters, context limits, and tools. Keep secrets on the server, validate requests, and isolate provider-specific code so models can be replaced.

Latency and throughput — Latency is elapsed time for one request; throughput is the amount processed per unit time. First-token latency matters to chat users, while full response time matters for jobs. Measure model inference separately from retrieval, reranking, network, and tool execution.

Rate limits and retries — Providers may limit requests, token usage, or concurrency. Use bounded queues and backoff when retrying transient errors. Repeating failed requests blindly can intensify outages and increase charges; sensitive or non-idempotent tool operations require stricter retry handling.

Streaming and caching — Streaming shows responses progressively but partial fragments may be incomplete or invalid. Validate the final output before triggering consequential actions. Cache safe reusable work such as parsed documents and embeddings; include user permissions, model versions, and settings in cache design.

Cost optimization — Account for prompt tokens, generated tokens, embeddings, storage, reranking, hosting, and monitoring. Reduce wasted context, reuse computed embeddings, batch compatible tasks, and route simple queries to smaller validated models. Measure cost per successful user task rather than per request alone.

Evaluation datasets — Create representative questions, reference outcomes, edge cases, and unacceptable-failure examples. Reserve a held-out test set. For RAG, evaluate whether search retrieved relevant evidence separately from whether the final answer is correct, complete, and faithful to that evidence.

Automated and human evaluation — Check schemas and exact fields with deterministic tests; use calibrated model-based grading for nuanced outputs and human review for subtle errors. Record disagreement between reviewers. A judge model can also hallucinate or favor certain responses, so its scores need validation.

Observability and tracing — Track request IDs, model and prompt versions, token usage, stage-level latency, retrieval results, tool calls, errors, and feedback. End-to-end traces reveal which subsystem failed. Redact sensitive documents and protect logs with appropriate retention and access limits.

Prompt injection — Malicious instructions hidden in retrieved documents, websites, or tool results may try to change model behavior or expose secrets. Treat external content as untrusted data, limit tool privileges, isolate credentials, verify actions, and test against adversarial examples. Prompt wording alone is insufficient defense.

Authorization and privacy — Authenticate users and enforce record-level or tenant-level access before retrieval. Apply data minimization, encryption, retention rules, and provider agreements appropriate to the application. Never assume the language model can protect confidential data solely because it was instructed to do so.

Tool safety and guardrails — Use narrow function schemas, server-side argument checks, least-privilege credentials, execution limits, and approval for consequential actions. Combine input rules, output validation, escalation, and policy checks. Guardrails must be evaluated against realistic failures rather than measured by filter count.

Fallbacks, delivery, and monitoring — Handle empty retrieval, invalid answers, unavailable providers, and tool failures with safe fallbacks or human escalation. Version models, prompts, datasets, and indexes; run regression tests and staged deployments. Monitor accuracy, latency, unsafe-action rate, user satisfaction, and cost after release.